目 录

一、引言
1.1 编写目的
1.2 软件概述
1.3 运行环境
1.4 术语与缩略语
二、软件总体设计
2.1 软件需求概括
2.2 总体架构设计
2.3 模块划分与关系
2.4 场景系统设计
2.5 主循环与帧更新
三、核心模块详细设计
3.1 游戏入口模块
3.2 渲染引擎模块
3.3 战斗引擎模块
3.4 随机爬塔局内管理模块
3.5 技能引擎模块
3.6 动画系统模块
3.7 输入处理模块
3.8 音频管理模块
3.9 新手引导模块
四、数据结构设计
4.1 棋盘数据结构
4.2 宠物数据结构
4.3 法宝数据结构
4.4 敌人数据结构
4.5 局内状态数据结构
4.6 持久化数据结构
4.7 全局加成(RunBuffs)数据结构
五、数据接口设计
5.1 本地存储接口
5.2 云数据库接口
5.3 云函数接口
5.4 排行榜接口
5.5 用户授权接口

六、出错处理设计
6.1 网络异常处理
6.2 数据异常处理
6.3 资源加载异常处理
6.4 运行时异常处理
七、性能优化设计
7.1 资源管理优化
7.2 渲染性能优化
7.3 内存管理优化
7.4 分包与加载优化

一、引言

1.1 编写目的

编写本设计说明书是软件开发过程的重要组成部分。本文档旨在详细描述深圳幸运呱科技有限公司灵宠消消塔小游戏软件（以下简称"本软件"）的软件架构设计、核心模块设计、数据结构设计、接口设计及出错处理设计，为软件著作权登记提供技术性文档依据。

本文档面向软件著作权审查人员，全面展示本软件的技术架构、设计思路和实现方案，证明本软件为独立开发的原创作品。

1.2 软件概述

本软件是一款基于微信小游戏平台的休闲策略游戏。游戏融合了随机爬塔、拖拽转珠消除和五行克制三大核心玩法元素，玩家通过在5x6的棋盘上拖拽珠子进行消除，触发对应属性宠物的攻击，逐层挑战越来越强大的敌人。游戏设有局外养成系统，包括修炼洞府（五维属性培养）、灵宠池（持久宠物收集与升星）和固定关卡挑战等内容。

本软件的主要功能包括:

* 五行转珠战斗系统: 金、木、水、火、土五种属性珠子，支持八方向拖拽交换和连锁消除
* 随机爬塔系统: 30层塔，每层随机生成战斗、奇遇、商店、休息等事件
* 宠物收集与养成: 100只五行属性宠物，支持星级提升和技能解锁
* 法宝装备系统: 50件法宝，提供各类全队被动增益效果
* 五行克制系统: 金克木、木克土、土克水、水克火、火克金的完整克制链
* 修炼洞府系统: 局外五维属性养成(体魄/灵力/悟性/根骨/神识)和修仙境界突破
* 灵宠池系统: 爬塔中满星宠物解锁入池，碎片升星培养持久灵宠
* 排行榜系统: 速通榜、图鉴榜、连击榜三个独立在线排行榜
* 云端存储与同步: 支持微信云开发的数据持久化和跨设备同步
* 新手引导系统: 4步渐进式教学引导
* 社交分享功能: 支持微信好友分享和群排行

软件主界面如下图所示:

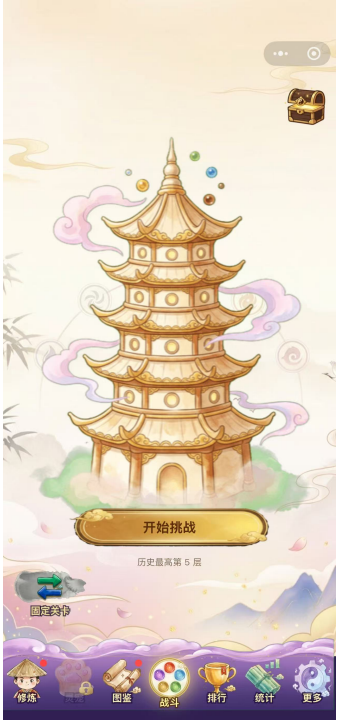

图1 游戏主界面 - 通天塔爬塔模式

游戏包含两种关卡模式: 通天塔(随机爬塔)和固定关卡。通天塔模式下玩家挑战30层随机生成的关卡，每层遭遇战斗、奇遇、商店、休息等随机事件，死亡后重新开始;固定关卡模式下玩家使用灵宠池中培养的持久灵宠编队出战，挑战固定难度的多波次战斗，获取高级灵宠和培养材料。固定关卡入口界面如下图所示:

图8 固定关卡入口 - 选择关卡挑战

1.3 运行环境

本软件的运行环境要求如下:

项目	要求
运行平台	微信小游戏
操作系统	iOS 10.0及以上 / Android 5.0及以上
微信版本	微信客户端 6.7.2 及以上
屏幕方向	竖屏(Portrait)
网络要求	需要网络连接(排行榜/云同步)，离线可单机游戏
开发语言	JavaScript (ES6+)
渲染技术	Canvas 2D
云服务	微信云开发
硬件要求	支持触摸屏的移动设备

1.4 术语与缩略语

本文档使用的术语和缩略语定义如下:

术语/缩略语	含义说明
随机爬塔	一种游戏模式，特点为随机生成关卡、永久死亡、每局重新开始
转珠	拖拽珠子进行位置交换，形成三连或以上同色消除的操作方式
五行	金、木、水、火、土五种属性元素，构成相互克制关系

Combo	连击，一次转珠操作中连续触发的消除组数
Canvas 2D	HTML5提供的二维图形绘制接口
DPR	Device Pixel Ratio，设备像素比
BGM	Background Music，背景音乐
SFX	Sound Effect，音效
CD	Cooldown，技能冷却时间
Buff	增益效果，对角色属性的临时或永久加成
BFS	Breadth First Search，广度优先搜索算法
Run	一局完整的随机爬塔游戏过程，从开局到结算
SDK	Software Development Kit，软件开发工具包
API	Application Programming Interface，应用程序接口

二、软件总体设计

2.1 软件需求概括

本软件采用模块化的软件设计方法，以微信小游戏框架为基础平台，使用原生JavaScript和Canvas 2D技术进行开发。软件采用单页面应用架构，通过场景状态机管理不同的游戏界面和交互逻辑。

本软件的核心需求包括以下几个方面:

* 高性能的实时渲染能力: 游戏需要60fps的流畅帧率，支持复杂的粒子特效和动画系统
* 精确的触摸输入处理: 支持拖拽、长按、滑动等多种触摸手势的识别与响应
* 可靠的数据持久化: 游戏进度、排行数据需要在本地和云端双重存储，确保数据安全
* 低延迟的战斗逻辑: 消除计算、伤害结算、连锁判定需要在单帧内完成
* 灵活的内容扩展性: 宠物、法宝、怪物等数据可独立配置，便于后续内容更新

2.2 总体架构设计

本软件采用"单例主控 + 场景驱动 + 委托分发"的架构模式。整体架构由一个Main主控类统一管理游戏状态和主循环，通过场景标识分发渲染和输入处理到各个功能模块。

架构核心设计要点:

* 单例Main: 所有游戏状态集中在Main类的实例(全局变量g)中，各模块通过接收g参数读写共享状态
* 场景驱动: g.scene控制当前场景，render()和onTouch()通过switch分发到对应的渲染和输入处理模块
* 委托模式: Main提供大量桩方法，内部转发到具体的引擎和视图模块执行
* 无事件总线: 模块间通过直接方法调用和共享全局对象通信，减少事件系统的复杂性

软件整体结构分为六大层次: 入口层(game.js)、主控层(main.js)、引擎层(engine/)、视图层(views/)、数据层(data/)和运行时层(runtime/)。各层之间职责清晰，通过主控层进行协调和调度。

系统整体模块信息如下:

模块层	模块名称	功能简述

入口层	game.js	游戏启动入口，创建Canvas，加载分包资源
主控层	main.js	主逻辑入口，Main类，管理全局状态和主循环
引擎层	battle.js	战斗引擎，消除算法、伤害计算、敌方回合
引擎层	skills.js	技能引擎，宠物技能触发、奖励应用
引擎层	runManager.js	随机爬塔局内生命周期管理
引擎层	animations.js	帧驱动动画更新系统
引擎层	tutorial.js	新手引导系统(4步固定棋盘引导)
视图层	render.js	渲染模块，Canvas 2D封装、图片缓存
视图层	screens.js	通用场景渲染(标题、结算、排行等)
视图层	battleView.js	战斗场景渲染(棋盘、敌人、血条)
视图层	prepareView.js	战前准备界面渲染
视图层	eventView.js	事件场景渲染
视图层	dialogs.js	弹窗系统(确认框、详情查看)
数据层	tower.js	塔层系统，五行属性，怪物生成，奖励池
数据层	pets.js	宠物数据(100只宠物、技能、星级)
数据层	weapons.js	法宝数据(50件法宝及被动效果)
数据层	storage.js	持久化存储(本地+云同步)，排行榜
输入层	touchHandlers.js	触摸输入，按场景分发触摸事件
运行时	music.js	音频管理(BGM、音效、连击递进)

2.3 模块划分与关系

软件的模块依赖关系采用树状结构，以main.js作为根节点，向下依赖各功能模块。模块依赖关系如下：

```
main.js (主控入口)
  +-- render.js ............. Render实例、主题色系统
  +-- data/storage.js ....... Storage(持久化+云同步+排行榜)
  +-- data/tower.js ......... 五行属性、事件生成、怪物、奖励池
  +-- data/pets.js .......... 宠物池、技能、星级配置
  +-- data/weapons.js ....... 法宝数据配置
  +-- runtime/music.js ...... MusicMgr(音频管理)
  +-- views/env.js .......... ViewEnv(共享渲染上下文)
  +-- views/screens.js ...... 通用场景渲染
  +-- views/battleView.js ... 战斗场景渲染
  +-- views/prepareView.js .. 战前准备渲染
  +-- views/eventView.js .... 事件场景渲染
  +-- views/dialogs.js ...... 弹窗系统
  +-- input/touchHandlers.js  触摸处理
  +-- engine/battle.js ...... 战斗引擎
  +-- engine/skills.js ...... 技能引擎
  +-- engine/animations.js .. 动画系统
  +-- engine/runManager.js .. 爬塔局内生命周期管理
  +-- engine/tutorial.js .... 新手引导
```

2.4 场景系统设计

游戏通过场景状态机管理界面切换，共包含13个场景。每个场景对应独立的渲染模块和触摸处理模块，通过全局变量g.scene进行切换控制。场景系统设计如下:

场景标识	功能描述	渲染模块	交互模块
loading	资源加载画面	screens	-
title	主菜单界面	screens	tTitle
prepare	战前准备(编辑阵容)	prepareView	tPrepare
event	地图事件展示	eventView	tEvent
battle	转珠战斗核心	battleView	tBattle
reward	战后三选一奖励	screens	tReward
shop	局内商店	screens	tShop
rest	休息回血/强化	screens	tRest
adventure	奇遇事件	screens	tAdventure
gameover	结算画面	screens	tGameover
ranking	在线排行榜	screens	tRanking
stats	个人战绩	screens	tStats
dex	灵兽图鉴	screens	tDex

场景切换通过修改g.scene全局变量实现，切换后render()和onTouch()方法会根据新的scene值自动分发到对应的渲染和输入处理函数。场景之间的数据传递通过g对象的共享属性完成，无需额外的事件通知机制。在切换到战斗场景前，系统会先进入prepare场景让玩家调整阵容。场景切换支持渐变过渡效果，通过fadeAlpha控制透明度动画，实现平滑的画面过渡。

各场景的生命周期包括进入(init)、运行中(update/render)和退出(cleanup)三个阶段。进入阶段完成数据初始化和UI准备，运行中阶段处理帧更新和用户交互，退出阶段清理临时资源。其中battle场景的生命周期最为复杂，涉及棋盘初始化、敌人生成、战斗状态机启动等多个步骤。场景渲染采用分层绘制策略，从底层背景到顶层UI依次叠加，确保视觉层次清晰。prepare场景负责让玩家在战前查看敌方信息、调整上场宠物顺序和更换法宝装备，支持拖拽排序和点击切换操作，确保玩家能够根据敌人属性合理配置队伍阵容。

战前准备场景(prepare)界面如下图所示，包含敌人信息、己方队伍、法宝背包和灵宠背包:

图2 战前准备界面 - 编辑阵容与装备

2.5 主循环与帧更新

游戏使用requestAnimationFrame驱动主循环，每帧依次执行update()和render()两个核心方法。update()负责状态更新，render()负责画面绘制。主循环使用try-catch包裹，确保单帧异常不会导致整个游戏停止。

主循环伪代码:

```
const loop = () => {
  this.af++   // 帧计数器
  try {
    this.update()   // 状态更新
    this.render()   // 画面渲染
  } catch(e) {
    console.error("loop error:", e)
  }
  requestAnimationFrame(loop)
}
```

update()方法的处理流程:

* 通用动画帧更新: 调用animations.updateAnimations(g)更新所有活跃动画
* 教学系统更新: 若教学模式活跃，调用tutorial.update(g)
* 消除动画处理: elimAnim状态下逐帧推进消除动画
* 下落动画处理: dropping状态下处理珠子下落填充
* 转珠超时检测: 检查dragTimer是否超过dragTimeLimit
* 宠物攻击展示: petAtkShow状态下显示数值翻滚动画

* 攻击结算: preAttack状态下执行实际伤害计算
* 敌方回合: enemyTurn状态下执行敌人攻击和技能
* 交换动画、战斗动画、HP变化动画的独立更新
* 排行榜自动刷新(每2分钟)

render()方法的处理流程:

* 清空画布: ctx.clearRect(0, 0, W, H)
* 屏幕震动偏移: 有震动时ctx.translate偏移
* 场景渲染: 根据当前scene分发到对应的渲染函数
* 全局飘字渲染: 伤害飘字dmgFloats和技能特效skillEffects
* 技能施放特效渲染
* 教学引导层渲染
* 宠物获得/升星弹窗渲染
* 满星庆祝画面渲染

三、核心模块详细设计

3.1 游戏入口模块

game.js是微信小游戏的启动入口文件，相当于传统程序的main函数。该模块负责以下核心功能:

* 平台检测: 自动识别微信/抖音运行环境，设置全局平台变量
* Canvas创建: 创建主屏幕Canvas，设置合适的分辨率和DPR
* 加载画面绘制: 在分包加载期间显示品牌Loading图
* 分包资源加载: 并行加载assets(图片资源)和audio(音频资源)两个分包
* 主逻辑启动: 所有分包加载完成后，require主逻辑模块main.js
* 侧边栏监听: 注册微信onShow事件，支持侧边栏复访功能

启动链路流程:

```
game.js (创建主Canvas、绘制loading背景)
  -> wx.loadSubpackage("assets") + wx.loadSubpackage("audio")
  -> 两个分包均加载完成后: require("./js/main")
    -> Main 构造函数
      -> 初始化 Storage、Render、ViewEnv
      -> 注册触摸事件
      -> 预加载关键图片(标题/按钮等)
      -> requestAnimationFrame(loop) 启动主循环
      -> 注册微信分享能力
```

3.2 渲染引擎模块

渲染模块(render.js)封装了Canvas 2D的底层绘制接口，提供图片缓存、主题色系统和屏幕适配功能。核心类为Render。

关键设计特点:

* 图片缓存系统: 使用_imgCache对象缓存已加载的wx.createImage()实例，getImg(path)方法实现懒加载并缓存，避免重复创建图片对象
* 预加载机制: preloadImages(paths, onProgress)返回Promise，支持加载进度回调，设置5秒超时保底确保不会无限等待
* 主题色系统: 全局TH对象定义完整的颜色方案(背景色bg、卡片色card、文字色text、强调色accent、危险色danger、成功色success等)，所有视图模块统一引用确保视觉一致性
* 屏幕适配方案: S = canvasWidth / 375缩放因子，以375pt宽度为设计基准，所有UI尺寸乘以S实现等比适配。safeTop处理iOS刘海屏安全区域
* ViewEnv共享: views/env.js提供ctx/R/TH/W/H/S/safeTop等渲染上下文的getter，供所有视图模块访问

属性配色系统设计:

属性	主色值	用途说明
金(Metal)	#ffd700	金属性珠子、伤害文字、属性标识
木(Wood)	#4dcc4d	木属性珠子、伤害文字、属性标识
土(Earth)	#d4a056	土属性珠子、伤害文字、属性标识
水(Water)	#4dabff	水属性珠子、伤害文字、属性标识
火(Fire)	#ff4d4d	火属性珠子、伤害文字、属性标识
心(Heart)	#ff69b4	心珠回血效果、回复数字

Render类还封装了常用的Canvas 2D绘制方法，包括圆角矩形绘制(roundRect)、文本截断(truncateText)、渐变填充(createLinearGradient)、阴影效果(setShadow)等。这些方法统一处理了DPR缩放和坐标转换，使得上层视图模块只需关注逻辑坐标，无需处理物理像素换算。

绘制管线还包括血条渲染(分段渐变+边框描边)、星级渲染(根据star数绘制填充/空心星星)、属性图标渲染(五行属性对应不同颜色和标识)等游戏专用绘制组件。所有绘制方法均通过ctx.save()/ctx.restore()保护绘图上下文状态，防止样式污染。文字绘制支持自动截断和省略号显示，确保长文本不会溢出预设区域。

3.3 战斗引擎模块

战斗引擎(battle.js)是游戏的核心模块，负责转珠消除逻辑、伤害计算、连击系统和敌方回合处理。采用有限状态机管理战斗流程。战斗界面如下图所示:

图3 战斗界面 - 5x6棋盘转珠消除

3.3.1 战斗状态机

战斗过程通过以下状态进行流转:

* playerTurn: 玩家拖拽转珠阶段，限时8秒(可被buff/法宝延长)
* elimAnim: 逐组消除动画阶段，每组16帧
* dropping: 珠子下落填充阶段，10帧后再检测连消
* petAtkShow: 宠物攻击数值翻滚展示，38帧
* preAttack: 攻击结算前等待，12帧
* preEnemy: 敌方回合前等待，30帧
* enemyTurn: 敌人攻击和技能释放
* victory: 胜利判定
* defeat: 失败判定(触发复活链)

状态机流转过程:

```
playerTurn --(松手/超时)--> elimAnim --(16帧/组)--> dropping
    ^                                    |
    |   <-- 有新的连消 <-----------------------------+
    |                                    |
    |   <-- preEnemy <- enemyTurn <- 无连消且combo=0 --+
    |                                    |
    |   <-- petAtkShow <- 无连消但combo>0 -------------+
    |         |
    |         v
    |     preAttack --> executeAttack --> settle
    |         |
```

```
|          v
+---- preEnemy --> enemyTurn --> playerTurn
```

3.3.2 消除算法

消除算法(findMatchesSeparate)采用三步流程实现高效的连通块检测:

* 横向扫描: 逐行查找连续3颗及以上同色珠子，标记匹配位置
* 纵向扫描: 逐列查找连续3颗及以上同色珠子，标记匹配位置
* BFS连通分组: 将所有标记的格子按属性做广度优先搜索，合并相邻同属性格子为连通块

算法返回值为数组，每个元素包含属性类型(attr)、消除数量(count)和格子坐标列表(cells)。防初始三连机制确保在initBoard()和fillBoard()中新生成的珠子不会立即形成三连。

3.3.3 伤害计算公式

战斗伤害计算采用多因子乘算模型:

```
最终伤害 = 基础攻击 x 消除倍率 x Combo倍率 x 全局加成
      x 属性加成 x 五行克制 x 暴击 x 残血爆发
      x 法宝加成 - 敌人防御
```

各因子详细说明:

计算因子	计算方式
基础攻击	getPetStarAtk(pet) x 消除倍率 x (1 + runBuffs.allAtkPct/100)
消除倍率	3消=1.0, 4消=1.5, 5消及以上=2.0
Combo倍率	1-8段:1+(combo-1)*0.35, 9-12段:+0.20/段, 13+段:+0.10/段
五行克制	克制x2.5, 被克x0.5
暴击	暴击率由技能/法宝/buff提供; 暴击伤害=基础50%+加成
残血爆发	HP<=15%时x2.0, HP<=30%时x1.5
防御抵扣	dmg = max(0, dmg - enemy.def)

3.3.4 棋盘管理

棋盘系统的核心设计参数:

参数	设定值	说明
棋盘尺寸	5行 x 6列	共30颗珠子
珠子类型	金/木/水/火/土/心	6种属性
操作方式	八方向拖拽交换	手指按住珠子拖动
操作限时	8秒	可被buff/法宝延长
灵珠权重	动态调整	敌人属性珠权重x1.4

3.4 随机爬塔局内管理模块

runManager.js负责随机爬塔局内的完整生命周期管理，包括开局初始化、层数推进、事件生成、奖励分发和局结束处理。

每局游戏开始时系统从全局宠物池中为玩家随机生成25只候选宠物(每属性5只)，并从中选取5只组成

初始队伍，同时分配1件初始法宝。随着层数推进，玩家通过战后奖励不断扩充宠物阵容和法宝装备，提升队伍的整体战斗能力。

核心接口函数:

函数名	功能描述
startRun(g)	新局初始化: 生成宠物池、初始队伍(5只)、初始法宝(1件)、重置runBuffs
nextFloor(g)	层数推进: 层数+1、境界成长(加血量上限)、每5层隐性加攻击、生成事件
endRun(g)	局结束: 更新最高层记录、清除存档、提交排行榜、跳转结算
saveAndExit(g)	暂存退出: 深拷贝局内全部状态到storage.savedRun，回到标题
resumeRun(g)	续玩恢复: 从存档恢复完整状态、清除存档、跳转事件场景
onDefeat(g)	失败处理: 临时复活->额外复活->法宝不灭金身->分享复活->真正失败

局内生命周期流程:

事件生成规则: 普通战斗基础权重70，15层后下降10，22层后再降10;精英战斗权重8，随层数递增;每10层(10/20/30)强制BOSS战;第5层强制精英战;奇遇、商店、休息事件从第4层开始出现。精英和BOSS战斗提供更高品质的奖励和更多经验值，是提升队伍实力的关键节点。

```
startRun(g)       新局初始化(宠物池、初始队伍、法宝)
  |
  v
nextFloor(g)      层数+1、境界成长、生成事件
  |
  v
curEvent 分发 -->  战斗 / 奇遇 / 商店 / 休息
  |
  v
奖励/强化 -------> nextFloor (循环)
  |
  v (floor > MAX_FLOOR 或 defeat)
endRun(g)         记录最高层、提交排行、清除存档
```

战斗胜利后的三选一奖励界面如下图所示，玩家可从宠物、法宝中选择一项奖励:

图4 奖励选择界面 - 战斗胜利三选一

3.5 技能引擎模块

技能引擎(skills.js)负责宠物技能的触发执行、战后奖励的应用、商店购买效果的执行以及奇遇和休息事件的效果处理。

核心接口函数:

函数名	功能描述
triggerPetSkill(g, pet, idx)	触发宠物主动技能，执行技能效果并重置CD
showSkillPreview(g, pet, idx)	长按显示技能预览浮窗信息
applyReward(g, rw)	应用战后三选一奖励(宠物/法宝/buff)
applyBuffReward(g, b)	应用buff奖励到runBuffs全局加成
applyShopItem(g, item)	应用商店购买物品的效果
applyRestOption(g, opt)	应用休息选项的效果(回血/强化)
applyAdventure(g, adv)	应用奇遇事件的随机效果

当宠物通过战后奖励获得重复副本时，会自动升星。升至满星(三星)时解锁终极形态和强化技能，同时完成图鉴收集。满星解锁界面如下图所示:

图6 灵宠满星解锁 - 终极形态与图鉴收集

3.6 动画系统模块

动画系统(animations.js)采用帧驱动方式，每帧调用update()更新所有活跃动画。系统支持多种动画类型:

动画类型	数据结构	更新方式
屏幕震动	g.shakeT/shakeI	每帧递减
Combo弹出	g._comboAnim	缩放弹跳+透明度渐隐
粒子特效	g._comboParticles[]	物理运动(速度+重力)+生命周期过滤
伤害飘字	g.dmgFloats[]	分三段: 停留->正常上飘->加速消失
技能特效	g.skillEffects[]	上飘+缩放弹跳+透明度渐隐
消除飘字	g.elimFloats[]	弹入->停留->上飘->消失
数值翻滚	g.petAtkNums[]	从0快速翻滚到最终值
交换动画	g.swapAnim	线性插值位移
HP变化	_enemyHpLoss等	从旧比例动画到新比例
敌人死亡	_enemyDeathAnim	爆裂粒子特效
Boss入场	_bossEntrance	帧数递减，缩放进场

3.7 输入处理模块

输入系统(touchHandlers.js)采用按场景分发的设计模式，每个场景有独立的触摸处理函数tXxx(g, type, x, y)。核心的转珠操作处理流程:

* touchstart: 判断触点落在棋盘格内，记录dragR/dragC/dragAttr等拖拽起始信息

* touchmove: 计算目标格位置，若为相邻格(八方向)且非封印状态，立即执行交换操作
* touchend: 调用g._checkAndElim()触发消除判定和连锁检测

宠物技能释放通过两种手势触发：上滑宠物头像触发技能释放(滑动距离超过阈值)，长按宠物头像300ms后显示技能预览浮窗。灵宠技能预览界面如下图所示，展示了灵宠的星级、攻击力、技能名称、效果描述和冷却回合数等信息:

图12 灵宠技能预览 - 青灵木鹿 · 春回大地

3.8 音频管理模块

音频管理模块(music.js)使用wx.createInnerAudioContext()管理音频实例，核心类为MusicManager，导出为单例MusicMgr。主要功能:

* BGM管理: 支持普通BGM和BOSS战BGM的切换播放
* 连击递进音效: playComboHit(comboNum)根据连击数动态调整音高和音量
* Combo里程碑音效: 在5/8/12/16连击时播放特殊音效
* 消除音效: playEliminate(count)按消除数量区分(3/4/5+消使用不同音效)
* 设置控制: 受settings.bgmOn和settings.sfxOn控制，支持独立开关

3.9 新手引导模块

新手引导系统(tutorial.js)提供4步渐进式教学，引导玩家学习核心操作。每步使用固定棋盘和弱敌，确保教学效果可控。

步骤	教学主题	教学内容	引导方式
第1步	转珠基础	引导L形拖拽凑3连金	起点到终点路径箭头
第2步	Combo与心珠	回合1教Combo连击, 回合2教心珠回血	切换引导

第3步	五行克制	回合1克制高伤, 回合2被克减伤	按回合切换引导
第4步	宠物技能	引导上滑释放宠物技能	上滑手势引导

引导系统通过canDrag(g)在教学中锁定非引导区域的拖拽操作，shouldEnemyAttack()控制教学中敌人是否攻击，onVictory(g)在教学胜利后自动切换到下一步或完成教学。

新手教学界面如下图所示，第1课首先以文字说明"消珠即攻击"的核心规则，引导玩家理解棋盘上消除三颗同色灵珠即触发对应属性灵宠发起攻击的基本机制:

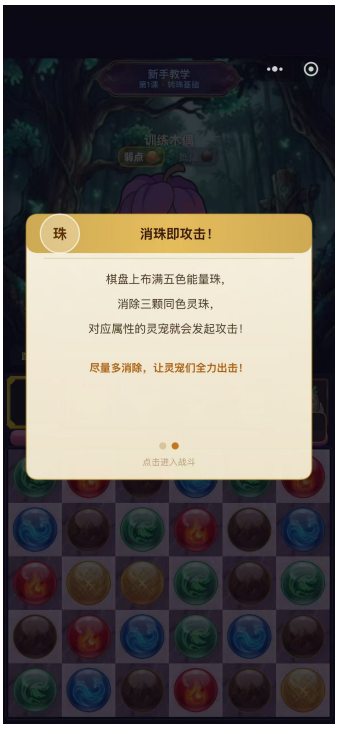

图9 新手教学 - 消珠即攻击规则说明

说明结束后进入实操阶段，玩家在固定棋盘上按引导路径拖拽珠子完成首次消除。教学使用弱敌(训练木偶)和固定棋盘布局，确保玩家能顺利体验完整的转珠战斗流程:

图10 新手教学 - 转珠实操引导

四、数据结构设计

4.1 棋盘数据结构

棋盘使用二维数组存储，每个格子包含属性和封印状态:

```
g.board[r][c] = {
  attr: "metal"|"wood"|"earth"|"water"|"fire"|"heart",
  sealed: false | number  // false=正常, number=封印剩余回合数
}
```

4.2 宠物数据结构

每只宠物包含基础属性、技能信息和星级数据:

```
{
  id: string,          // 唯一标识, 如 "metal_1"
  name: string,        // 显示名
  attr: string,        // 属性(金木水火土)
  atk: number,         // 基础攻击力
  star: 1|2|3,         // 星级(1-3)
  currentCd: number,   // 当前技能CD剩余回合
  skill: { ... },      // 技能数据(2星解锁)
  skill3: { ... },     // 3星强化技能(可选)
  tier: "T1"|"T2"|"T3", // 档位等级
  avatar: string,      // 头像资源路径
}
```

4.3 法宝数据结构

法宝数据包含效果类型、稀有度和数值参数:

```
{
  id: string,
  name: string,
  type: string,      // 效果类型(attrDmgUp/allAtkUp/revive等)
  desc: string,      // 效果描述
  rarity: "normal"|"rare"|"epic", // 稀有度
  attr: string,      // 关联属性(部分法宝)
  pct: number,       // 百分比效果值
  val: number,       // 固定值效果
}
```

战斗中长按法宝图标可查看法宝详情，包括法宝名称、效果描述和被动触发说明。法宝为被动效果，装备后全程自动生效，无需手动操作。法宝详情界面如下图所示:

图13 法宝详情界面 - 万寿青莲被动效果

4.4 敌人数据结构

敌人数据包含战斗属性、技能列表和资源路径:

```
{
  name: string,
  attr: string,      // 五行属性
  hp: number,
  maxHp: number,
  atk: number,
```

```
  def: number,
  skills: string[],     // 技能key列表
  avatar: string,       // 立绘路径
  battleBg: string,     // 战斗背景(精英/BOSS)
  isElite: boolean,
  isBoss: boolean,
}
```

4.5 局内状态数据结构

随机爬塔局内状态包含当前层数、上场宠物、装备法宝、各类buff加成和一次性标记等完整的局内信息:

```
{
  floor: number,              // 当前层数
  pets: Pet[],                // 上场5只宠物
  weapon: Weapon|null,        // 当前装备法宝
  petBag: Pet[],              // 宠物背包
  weaponBag: Weapon[],        // 法宝背包
  sessionPetPool: object,     // 本局宠物池(每属性5只)
  heroHp: number,             // 当前生命值
  heroMaxHp: number,          // 最大生命值
  heroShield: number,         // 护盾值
  heroBuffs: Buff[],          // 战斗内临时buff
  runBuffs: RunBuffs,         // 局内全局加成
  curEvent: object,           // 当前层事件
  realmLevel: number,         // 修仙境界等级
}
```

4.6 持久化数据结构

跨局保留的持久化数据，同时存储在本地和云端:

```
{
  bestFloor: number,          // 历史最高层数
  totalRuns: number,          // 总挑战次数
  stats: {
    totalBattles: number,     // 总战斗次数
    totalCombos: number,      // 总连击次数
    maxCombo: number,         // 最高连击记录
    bestFloorPets: [],        // 最高层阵容
    bestFloorWeapon: object, // 最高层法宝
    bestTotalTurns: number,  // 最快通关回合数
  },
  settings: {
    bgmOn: boolean,           // 背景音乐开关
    sfxOn: boolean,           // 音效开关
  },
  petDex: string[],           // 图鉴: 已收集3星宠物ID
  savedRun: object|null,      // 局内暂存(退出续玩)
```

```
}
```

局外养成系统中，灵宠池是玩家长期收集和培养灵宠的核心功能。在随机爬塔模式中将宠物升至满星(三星)后自动解锁入池，成为持久灵宠。灵宠池界面支持按五行属性筛选，展示每只灵宠的星级、等级和攻击力。灵宠池界面如下图所示:

图14 灵宠池界面 - 持久灵宠收集与管理

点击灵宠池中的灵宠可查看详细信息，包括攻击力计算公式(基础+等级+星级倍率)、技能效果、能力提升进度和碎片升星状态。灵宠通过经验升级提升等级和攻击力，通过碎片提升星级解锁更强形态。灵宠详情界面如下图所示:

图15 灵宠详情界面 - 冰玄灵蛾属性与升级

修炼洞府是另一个重要的局外养成系统，玩家在修炼界面中可查看当前境界等级、修炼点数以及五维属性(体魄/灵力/悟性/根骨/神识)的成长情况。境界从凡人开始，随着等级提升逐步突破到更高境界(感气期、练气期、筑基期等)，解锁更多属性加成。修炼洞府界面如下图所示:

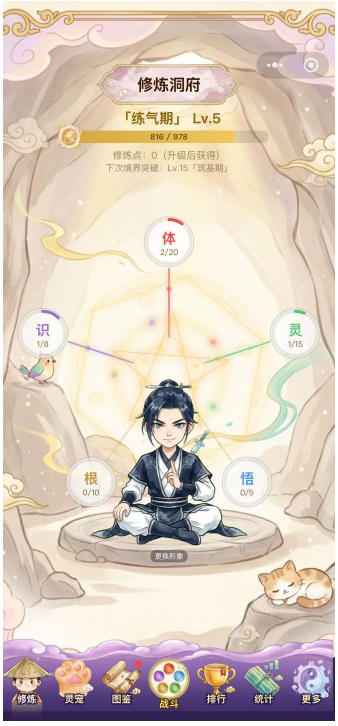

图5 修炼洞府界面 - 境界成长与五维属性

4.7 全局加成(RunBuffs)数据结构

RunBuffs是随机爬塔局内的全局加成系统，记录玩家在本局中获得的各类永久增益效果。这些加成来源于战后奖励、商店购买、奇遇事件和休息选项，在整局游戏中持续生效。

```
{
  allAtkPct: number,       // 全属性攻击加成百分比
  allDmgPct: number,       // 全伤害加成百分比
  attrDmgPct: {            // 各属性独立伤害加成
    metal, wood, earth, water, fire: number
  },
  heartBoostPct: number,   // 心珠回复加成百分比
  weaponBoostPct: number,  // 法宝效果加成百分比
  extraTimeSec: number,    // 转珠额外时间(秒)
  hpMaxPct: number,        // 血量上限加成百分比
  comboDmgPct: number,     // Combo伤害加成百分比
  elim3DmgPct: number,     // 3消伤害加成
  elim4DmgPct: number,     // 4消伤害加成
  elim5DmgPct: number,     // 5消伤害加成
  counterDmgPct: number,   // 克制伤害加成百分比
  skillDmgPct: number,     // 技能伤害加成百分比
  regenPerTurn: number,    // 每回合自动回血
  dmgReducePct: number,    // 受伤减免百分比
  bonusCombo: number,      // 额外Combo加成
  stunDurBonus: number,    // 眩晕持续加成
  extraRevive: number,     // 额外复活次数
}
```

五、数据接口设计

5.1 本地存储接口

本地存储使用微信小游戏的wx.setStorageSync/wx.getStorageSync接口，存储key为"wxtower_v1"。每次数据变更时即时写入本地存储，确保数据不丢失。

5.2 云数据库接口

云数据库使用微信云开发环境(cloud1-6g8y0x2i39e768eb)，采用双重存储策略，本地存储和云数据库同步运行。

存储架构:

```
        +----------------+
        |   Storage 类   |
        +---+--------+---+
            |        |
  +---------v--+  +--v----------+
  | 本地存储    |  | 云数据库     |
  | wx.storage  |  | playerData  |
  | (即时写入)  |  | (2秒防抖)   |
  +------------+  +-------------+
```

写入策略: 每次_save()先写本地，再以2秒防抖间隔同步到云端。冲突处理: _syncFromCloud()比较_updateTime时间戳，云端较新时深度合并到本地;数值字段智能合并(bestFloor取大值，bestTotalTurns取小值)。

5.3 云函数接口

系统包含以下4个云函数:

云函数名	功能描述	调用时机
getOpenid	获取用户openid	Storage初始化时
initCollections	创建云数据库集合	Storage初始化时
ranking	排行榜提交与查询(6种action)	局结束提交 + 排行榜页拉取
resetTaskAndWeekly	每日/每周任务重置	定时触发

云函数调用采用统一的错误处理策略: 所有调用均使用try-catch包裹，调用失败时静默降级，不影响游戏核心流程;getOpenid在Storage初始化时调用，获取成功后缓存到本地，后续不再重复调用;ranking云函数支持批量提交和分页查询，单次最多返回50条记录。

5.4 排行榜接口

系统实现三个独立的在线排行榜:

榜单名称	云集合	排序规则	提交时机
速通榜	rankAll	floor降序, totalTurns升序	局结束时
图鉴榜	rankDex	petDexCount降序	局结束时/独立提交
连击榜	rankCombo	maxCombo降序	局结束时/独立提交

排行榜缓存策略: 30秒客户端缓存(rankLastFetch)避免频繁调用云函数;后台预热在Storage初始化完成后静默拉取速通榜;排行榜页面每2分钟自动刷新。

排行榜云函数支持6种action: submitAll(提交速通记录)、submitDex(提交图鉴记录)、submitCombo(提交连击记录)、getAll(获取速通榜)、getDex(获取图鉴榜)、getCombo(获取连击榜)。每次提交时附带玩家昵称和头像URL用于排行榜显示。排行榜展示默认显示前50名，超出范围显示玩家自身排名和数据。排行榜数据通过微信云开发的aggregation聚合查询实现高效的排序和分页检索。排行榜页面支持速通榜、图鉴榜和连击榜之间的Tab切换，各榜单独立缓存和刷新。

玩家可在"我的战绩"界面查看个人统计数据，包括通关状态、最速通关回合数、总挑战次数、总战斗场次、图鉴收集进度、最高连击记录以及最高记录阵容等信息。战绩界面如下图所示:

图7 我的战绩界面 - 个人数据统计与记录

5.5 用户授权接口

用户授权接口用于获取微信用户昵称和头像信息，主要用于排行榜显示。授权流程采用微信原生User InfoButton方式实现。

授权流程设计:

* 使用wx.createUserInfoButton创建透明按钮，覆盖在Canvas排行按钮位置上
* 用户点击后触发微信授权弹窗，获取用户允许后保存昵称和头像URL
* 过滤无效默认值(如"微信用户")，仅保存有效的用户信息
* 授权失败时引导用户到微信设置页重新开启权限
* 授权状态通过userAuthorized标记，控制按钮的显示和隐藏
* 原生按钮位置需从Canvas像素换算回CSS像素(除以DPR)，确保覆盖准确

六、出错处理设计

6.1 网络异常处理

由于游戏支持离线单机游玩，网络异常不会导致游戏崩溃。具体处理策略:

* 云函数调用失败: 使用try-catch捕获，失败时静默降级，不影响游戏进行
* 云同步失败: 本地存储作为主存储，云端同步失败后2秒防抖重试，多次失败后放弃本次同步
* 排行榜拉取失败: 显示上次缓存的数据，或显示"暂无数据"提示
* 网络超时: 统一设置5000ms超时，超时后自动放弃请求

6.2 数据异常处理

数据层面的异常处理措施:

* 存档损坏: wx.getStorageSync失败时重建默认数据，不影响新游戏开始
* 云端数据冲突: 通过_updateTime时间戳比较，智能合并而非简单覆盖
* 宠物/法宝ID无效: 数据访问时做空值检查，无效ID跳过处理
* 配置数据缺失: 使用默认值兜底，确保游戏逻辑完整性

6.3 资源加载异常处理

图片和音频资源加载异常的处理:

* 图片加载失败: getImg()返回null，渲染时跳过绘制该图片，不影响游戏流程
* 预加载超时: preloadImages设置5秒超时保底，超时后仍继续游戏初始化
* 分包加载失败: game.js中对分包加载设置fail回调，记录错误日志
* 音频创建失败: MusicManager在createInnerAudioContext失败时静默处理，不播放音效不影响游戏

6.4 运行时异常处理

主循环和运行时异常的处理:

* 主循环异常: loop函数中使用try-catch包裹update()和render()，捕获异常后输出错误日志并继续下一帧
* 触摸事件异常: onTouch处理函数中捕获异常，防止一次触摸错误导致后续触摸失效
* 战斗逻辑异常: 消除算法和伤害计算中对边界条件做防御性编程，确保不会出现除零或数组越界
* 状态异常恢复: 当检测到状态异常(如battle阶段无敌人)时自动重置到安全状态

七、性能优化设计

7.1 资源管理

资源管理的优化措施:

* 图片缓存: 使用_imgCache对象缓存已加载的图片实例，避免重复创建wx.createImage()对象
* 懒加载: getImg(path)采用懒加载策略，首次访问时创建图片对象并缓存
* 分包加载: 将大体积的图片资源和音频资源分别放入assets和audio分包，主包体积控制在合理范围内
* 预加载优先级: 启动时仅预加载6张关键UI图片(标题/按钮等)，其余图片在使用时按需加载

7.2 渲染优化

渲染层面的优化策略:

* DPR适配: canvas.width = windowWidth * pixelRatio，确保高分辨率屏幕显示清晰
* 等比缩放: S = canvasWidth / 375缩放因子，所有坐标乘以S，一套代码适配各种屏幕尺寸

* 安全区域: safeTop处理iOS刘海屏，UI内容从安全区域下方开始绘制，避免被刘海遮挡
* 帧率控制: 使用requestAnimationFrame驱动主循环，跟随系统刷新率，不会过度消耗CPU

7.3 内存优化

内存使用的优化方案:

* 数组复用: 飘字、粒子等高频创建的对象在生命周期结束后从数组中过滤移除，释放内存
* 图片管理: 图片缓存统一管理，避免同一张图片创建多个Image对象
* 状态清理: 局结束时(endRun)清理所有局内状态和临时对象，防止内存泄漏
* 音频复用: 音频实例通过MusicManager统一管理，不使用时及时销毁，避免音频上下文数量超限

7.4 分包与加载优化

微信小游戏对主包体积有严格限制，本软件采用分包策略将资源和代码合理拆分。分包配置如下:

分包名	根目录	包含内容	加载时机
主包	/	game.js、js/、loading_bg.jpg	启动即加载
assets	assets/	全部图片资源(宠物、怪物、背景、UI等)	主包加载后并行加载
audio	audio/	BGM和音效文件	主包加载后并行加载
audio_bgm	audio_bgm/	背景音乐文件	主包加载后并行加载

加载优化策略:

* 并行加载: assets和audio分包同时发起加载请求，缩短总加载时间
* Loading画面: 在分包加载期间显示品牌Loading图，提升用户等待体验
* 加载完成回调: 所有分包加载完成后才执行require主逻辑，确保资源就绪
* 失败重试: 分包加载失败时自动重试，超过重试次数后提示用户检查网络
* 关键图片预加载: 进入游戏前预加载标题画面、按钮等关键UI图片，确保首屏体验流畅